

category: dotnet-general

tags: [ProjectReference, transitive-packagereference, System.Drawing.Common, csproj, refactor, silent-breakage]

severity: high

created: 2026-07-20

updated: 2026-07-20

project-origin: iPGSv4

## Đổi ProjectReference sang bộ thư viện mới làm rớt type ngầm không báo lỗi rõ ràng ngay

### Tình huống gặp phải

User đổi `ProjectReference` của `IPGS.Object.csproj` từ bộ project cũ (`0.BaseLIB\Kztek.Api`, `Kztek.Object.Entity`, `Kztek.Tool`) sang bộ project mới cross-platform (`Kztek.Api.MultyPlatform`, `Kztek.Object.MultyPlatform`, `Kztek.Tool.MultyPlatform`), đồng thời xoá luôn `<PackageReference Include="RestSharp.Authenticators.Digest">`, `Newtonsoft.Json`, `RestSharp` trực tiếp trong `IPGS.Object.csproj` (dựa vào transitive từ project mới) — và vô tình xoá luôn `<ProjectReference Include="..\iParkingv5.Lpr\...">` trong cùng lần sửa `ItemGroup`.

### Triệu chứng / Lỗi

```
error CS0234: 'Digest' does not exist in namespace 'RestSharp.Authenticators'
error CS0234: 'LprDetector' does not exist in namespace 'iParkingv5'
error CS0246: 'ILpr' could not be found
error CS1069: type 'Image' forwarded to assembly 'System.Drawing.Common' - thiếu
reference
```

### Nguyên nhân gốc rễ (Root Cause)

1. **Transitive PackageReference chỉ chảy 1 chiều theo hướng ProjectReference** (A reference B → A thừa hưởng package của B), KHÔNG chảy ngược. Xoá `ProjectReference` cũ đồng nghĩa xoá luôn mọi type/package transitive nó từng mang lại — kể cả những type không liên quan trực tiếp tới lý do đổi reference (ở đây: `ILpr` từ `iParkingv5.Lpr` bị rớt theo dù mục tiêu chỉ là đổi `BaseLIB` → `MultyPlatform`).
2. `'RestSharp.Authenticators.Digest'` là **package add-on riêng**, không nằm trong package `RestSharp` chính — project mới (`Kztek.Api.MultyPlatform`) có `RestSharp` nhưng KHÔNG có gói `Digest` add-on, nên digest auth cho camera Dahua/HIK vẫn phải khai báo trực tiếp trong `IPGS.Object.csproj`.
3. `'System.Drawing.Common'` không được tự forward trên `net8.0` thường (không `-windows`) nếu không có project/package nào trong đồ thị dependency của chính project đó mang nó vào — dù project TIÊU THỤ (consumer) có gói này, project bị tiêu thụ (dependency) không tự nhiên "mượn" được.

4. **Lỗi build chỉ hiện đúng những gì trực tiếp bị ảnh hưởng** — sửa xong 1 lỗi (VD thêm lại `ProjectReference iParkingv5.Lpr`) có thể kéo theo lỗi MỚI ở tầng xa hơn (ở đây: `iParkingv5.Lpr` lại kéo vào `0.BaseLIB\Kztek.Object.Entity` — một project HOÀN TOÀN KHÁC, thiếu `System.Drawing.Common` của chính nó) — phải build lặp lại nhiều vòng, không thể đoán hết lỗi từ 1 lần build đầu.

## Giải pháp

1. Khi đổi `ProjectReference` (không chỉ thêm, mà XOÁ project cũ), luôn `grep` toàn bộ using/type trong file đang refactor để liệt kê hết type đến từ project cũ trước khi xoá — không chỉ sửa theo lỗi build đầu tiên hiện ra.
2. Add-on package con (Digest, .Extensions, .Windows...) không tự đi kèm package chính khi đổi major version hoặc đổi nguồn — luôn kiểm tra riêng.
3. Build lặp lại nhiều vòng (`dotnet build <project>.csproj` từng project một, từ lá tới gốc) thay vì build cả solution 1 lần — lỗi tầng sâu (project ngoài repo, project transitive 2-3 cấp) dễ bị che khuất nếu chỉ nhìn tổng số lỗi cuối cùng.
4. Nếu 1 project transitive nằm NGOÀI repo hiện tại (VD `0.BaseLIB`) bị lộ lỗi do thay đổi trong repo này, vẫn có thể sửa (thêm package thiếu) nếu đó là fix an toàn, tối thiểu (thêm 1 dòng `PackageReference`) — nhưng phải báo rõ cho user vì đang sửa ra ngoài phạm vi repo đang làm việc.

## Áp dụng lại (How to reuse)

- Khi thấy `error CS0234/CS0246` ngay sau khi ai đó đổi `ProjectReference` → luôn nghi ngờ transitive dependency bị rút trước, không vội thêm `using` hay implement lại type.
- Khi 1 project library "đột nhiên" thiếu `System.Drawing/Image/Graphics` sau khi đổi reference → kiểm tra xem project đó tự có `System.Drawing.Common` package hay đang dựa transitive từ project khác đã bị đổi/xoá.
- Build từng project theo thứ tự lá→gốc để cô lập lỗi, tránh 1 lần build solution ném ra hàng trăm dòng lỗi khó phân loại.

## Chú ý / Cạm bẫy (Gotchas)

- ⚠ Đừng giả định "build solution 1 lần là thấy hết lỗi" — lỗi tầng transitive 2-3 cấp (project ngoài repo) chỉ lộ ra SAU KHI đã fix xong lỗi tầng 1.
- ⚠ Khi phạm vi sửa lan sang project ngoài repo đang làm việc (khác `.git root`), PHẢI báo rõ cho user trước/sau khi sửa — không âm thầm coi là "cùng 1 task".

## Tham chiếu

- Project liên quan: `iPGSv4 (IPGS.Object.csproj, iParkingv5.Lpr.csproj)`, `0.BaseLIB\Kztek.Object\Kztek.Object.Entity.csproj`